MCGI: Manifold-Consistent Graph Indexing for Billion-Scale Disk-Resident Vector Search

Dongfang Zhao
Tacoma School of Engineering and Technology
Paul G. Allen School of Computer Science & Engineering
University of Washington, USA
dzhao@cs.washington.edu

Abstract

Graph-based Approximate Nearest Neighbor (ANN) search often suffers from performance degradation in high-dimensional spaces due to the Euclidean-Geodesic mismatch, where greedy routing diverges from the underlying data manifold. To address this challenge, we propose Manifold-Consistent Graph Indexing (MCGI), a geometry-aware and disk-resident indexing method that leverages Local Intrinsic Dimensionality (LID) to dynamically adapt search strategies to the intrinsic geometry of the data. Unlike standard algorithms that treat dimensions uniformly, MCGI modulates its beam search budget based on in-situ geometric analysis, which eliminates the dependency on static hyperparameters. Theoretical analysis confirms that MCGI provides robust approximation guarantees by preserving manifold-consistent topological connectivity. Extensive evaluations against five industry-standard baselines across five datasets, ranging from million to billion scales, demonstrate the superiority of our approach: MCGI achieves 5.8x higher throughput at 95% recall on the high-dimensional GIST1M dataset compared to the state-of-the-art; on the billion-scale SIFT1B and T2I-1B datasets, MCGI demonstrates its scalability by reducing high-recall query latency by 3.0x while maintaining performance parity on low-dimensional benchmarks.

1 Introduction

The advent of Large Language Models [4, 39] has fundamentally transformed the landscape of information retrieval and knowledge management. To address the inherent limitations of LLMs, such as hallucinations [19] and knowledge cutoff dates, Retrieval-Augmented Generation (RAG) [27] has emerged as a critical architectural paradigm. RAG relies heavily on the ability to retrieve semantically relevant context from massive corpora in real-time, typically via dense vector representations [24]. This dependency has placed Approximate Nearest Neighbor Search (ANNS) at the core of modern data infrastructure, demanding vector indices that can scale to billion-point datasets [35] while maintaining low latency and high recall under strict production constraints.

State-of-the-art ANNS solutions have noticeably converged on graph-based indices, with DiskANN (Vamana) [36] being a representative example for large-scale, SSD-resident workloads. These algorithms typically employ greedy routing on a proximity graph to navigate from an entry point to the query target. While such methods exhibit exceptional performance on standard benchmarks like SIFT1M [21] (128 dimensions), their efficiency degrades significantly in high-dimensional spaces, such as GIST1M (960 dimensions). This degradation is often attributed to the curse of dimensionality [2], where the distance contrast diminishes, and the Euclidean shortest path on the graph diverges from the geodesic path on the underlying data manifold . We refer to this phenomenon as the *Euclidean-Geodesic mismatch*. When the routing algorithm ignores the intrinsic geometry of the data, it performs excessive backtracking and disk I/O, rendering the search inefficient, which is

Preprint.

particularly problematic in high-recall regimes where production systems usually expect Recall@10 $\geq 95\%$.

Our key insight is that high-dimensional real-world data is rarely uniformly distributed. Instead, it typically adheres to the Manifold Hypothesis [37, 33], residing on lower-dimensional structures embedded within the ambient space. Consequently, the search difficulty is not uniform across the dataset but is modulated by the Local Intrinsic Dimensionality (LID) [26]. In regions where the data manifold is flat (low LID), greedy routing is effective; however, in regions with high curvature or complex topology (high LID), standard greedy strategies fail to identify the correct descent direction. *We thus argue that an optimal indexing strategy must be manifold-aware, dynamically allocating computational resources based on the local geometric complexity.*

To achieve that, we introduce Manifold-Consistent Graph Indexing (MCGI), a geometry-aware disk-based indexing architecture designed to align Euclidean search with the underlying data manifold. By integrating LID estimation directly into the routing logic, MCGI adapts its traversal strategy to the local topology of the data.

Our contributions are as follows:

- We establish a theoretical method linking local intrinsic dimensionality to graph navigability, offering a rigorous justification for adaptive beam search on non-Euclidean manifolds.
- We develop an adaptive routing algorithm that dynamically modulates the search budget based on real-time geometric analysis. This design eliminates the dependency on static and manually tuned hyperparameters that often limit the adaptability of existing methods.
- We conduct comprehensive empirical evaluations against five industry-standard baselines across five datasets ranging from million to billion scales. Results demonstrate that MCGI achieves up to 5.8x higher throughput on high-dimensional vectors (e.g., 960-dimensional GIST1M) and reduces query latency by 3x on the billion-scale SIFT1B compared to the state-of-the-art baselines, confirming its robustness in mitigating both I/O and computational bottlenecks across diverse production environments.

2 Related Work

Vector Indexing Paradigms While traditional sparse retrieval methods rely on lexical matching [32], memory resident graph indices like HNSW [29] establish state-of-the-art dense retrieval but face billion-scale memory bottlenecks. Disk-based approaches like DiskANN [36] and SPANN [5], along with predecessors like NSG [9], adapt topologies or inverted structures for SSDs. Recent advancements such as CSPG [41] further optimize sparse proximity graphs to reduce unpromising explorations, yet these methods predominantly rely on static routing parameters evaluated on moderate dimensionality. Unlike these static paradigms, MCGI dynamically modulates the search budget using the estimated LID to align graph traversal with the underlying manifold structure.

High-Dimensional Indexing Early space partitioning trees like the KD-tree [3] and R-tree [14] severely degrade to linear scan performance in dimensions higher than 20 due to the curse of dimensionality. Approximate methods address this through sub-linear hashing like LSH [17, 7], which incurs excessive storage overhead, or subspace quantization like PQ [21] and OPQ [10], and randomized structures like RP-Trees [6], which sacrifice accuracy through compression. Furthermore, recent literature extensively studies the theoretical limits of navigable graphs [8] and the efficiency of index construction [43] in strictly high-dimensional regimes. MCGI circumvents aggressive quantization losses and redundant hashing overheads by preserving the exact connectivity benefits of graph traversal adapted to local geometry.

Intrinsic Dimensionality Theoretical metrics such as the doubling dimension [13] and expansion dimension [23] provide asymptotic bounds on search complexity, and recent analyses establish worst-case performance limits for popular implementations like DiskANN and HNSW based on intrinsic dimensionality [18]. Maximum likelihood estimators [26] for Local Intrinsic Dimensionality enable robust practical estimations . Existing works typically utilize LID passively for query hardness prediction [15, 40] or detecting adversarial examples [28] without altering the underlying index structure. Diverging from passive descriptive applications, MCGI actively utilizes LID as a prescriptive control signal to dynamically modulate graph traversal parameters for efficient routing.

3 Methodology

3.1 Notations and Definitions

We start by briefly introducing the notions of Local Intrinsic Dimensionality (LID) in the words of analysis. A full treatment can be found, for example, in Houle [16].

Definition 3.1 (Local Intrinsic Dimensionality)**.** Let $\mathcal{X}$ be a domain equipped with a distance measure $d : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^+$. For a reference point $x \in \mathcal{X}$, let $F_x(r) = \mathbb{P}(d(x, Y) \leq r)$ denote the cumulative distribution function (CDF) of the distance between x and a random variable Y drawn from the underlying data distribution, where $\mathbb{P}$ denotes the probablity function and r denotes the radius. The Local Intrinsic Dimensionality (LID) of x, denoted as $\mathrm{LID}(x)$, is defined as the intrinsic growth rate of the probability measure within the neighborhood of x:

$$\mathrm{LID}(x) \triangleq \lim_{r \to 0} \frac{r \cdot F'_x(r)}{F_x(r)} = \lim_{r \to 0} \frac{d \ln F_x(r)}{d \ln r}, \tag{1}$$

provided that the limit exists and $F_x(r)$ is continuously differentiable for $r > 0$.

Remark 3.2 (Institution of LID). The definition of LID can be understood as a measure of the multiplicative growth rate of the volume of a ball centered at x with radius r as r approaches 0. Let D denote the dimensionality of the ambient space. If the data lies on a local D-dimensional manifold, then the CDF around an infinitely small neighborhood of x satisfies:

$$F_x(r) \approx C \cdot r^D, \tag{2}$$

where C is a constant. Thus, the following holds:

$$F'_x(r) \approx C \cdot D \cdot r^{D-1}. \tag{3}$$

Combining equations (2) and (3), we get:

$$D \approx \frac{F'_x(r)}{F_x(r)} \cdot r, \tag{4}$$

thus Eq. (1).

While Eq. (1) provides an intuitive closed-form formula for intrinsic dimensionality, in practice we usually do not have access to the true CDF $F_x(r)$. Instead, we can estimate LID from a finite sample of distances from x to its neighbors using Maximum Likelihood Estimation (MLE), as proposed by [26]. According to [1], LID can be estimated as follows.

Definition 3.3 (LID Maximum Likelihood Estimator)**.** Given a reference point x and its k-nearest neighbors determined by the distance measure d. Let $r_i = d(x, v_i)$ denote the distance to the i-th nearest neighbor, sorted such that $r_1 \leq \cdots \leq r_k$. Following the formulation in [1], which adapts the Hill estimator for intrinsic dimensionality, the LID at x can be estimated as:

$$\widehat{\mathrm{LID}}(x) = -\left(\frac{1}{k} \sum_{i=1}^{k} \ln \frac{r_i}{r_k} \right)^{-1}. \tag{5}$$

Remark 3.4 (Intuition behind the LID Estimator). The rationale behind this estimator stems from modeling the local distance distribution as a power law. Within a sufficiently small neighborhood bounded by r_k, the cumulative distribution of distances $F_x(r)$ grows proportionally to r^m, where m is the intrinsic dimensionality. By treating the observed normalized distances r_i/r_k as independent samples drawn from this power law distribution, we can formulate a likelihood function. Solving for the MLE of the exponent m yields exactly the negative inverse of the average log ratios. This elegantly translates the abstract continuous growth rate into a discrete, computationally efficient statistical average.

3.2 Mapping Function from LID to Pruning Parameter

The primary hypothesis of the proposed Manifold-Consistent Graph Indexing (MCGI) is that the graph topology should adapt to the local geometric complexity, rather than being a static vertex-edge structure. While the following formulation may seem technical, MCGI is conceptually straightforward: it uses the estimated LID to determine how aggressively to prune edges during graph

construction. That is, in regions where the Local Intrinsic Dimensionality (LID) is low, the data manifold approximates a flat Euclidean subspace. In such isotropic regions, the Euclidean metric is a reliable proxy for geodesic distance, allowing for aggressive edge pruning (larger α in [36]) to permit long-range direct connections without risking semantic shortcuts. Conversely, regions with high LID typically exhibit significant curvature, noise, or singularity. In this case, the Euclidean distance often violates the manifold geodesic structure. To preserve topological fidelity, the indexing algorithm must adopt a conservative pruning strategy (smaller α in [36]), thereby forcing the search to take smaller, safer steps along the manifold surface. The question then becomes how to quantitatively map the estimated LID to the pruning parameter α that controls the edge selection during graph construction. We refer to such a function as the *mapping function* $\Phi : \mathbb{R}^+ \to [\alpha_{\min}, \alpha_{\max}]$, which translates the geometric complexity of the local neighborhood into a corresponding pruning constraint.

Let $u \in V$ be a node in the graph and $\widehat{\text{LID}}(u)$ be its estimated LID. We define the *pruning parameter* $\alpha(u)$ as:

$$\alpha(u) \triangleq \Phi(\widehat{\text{LID}}(u)), \tag{6}$$

where Φ will be constructed to satisfy the following geometric requirements: in regions with high LID, the graph should enforce a stricter connectivity constraint (smaller α) to avoid short-circuiting the manifold; conversely, in low-LID regions, the constraint can be relaxed (larger α).

To ensure the mapping is robust across datasets with varying complexity scales, we employ Z-score normalization based on the empirical distribution of the LID estimates. We first compute the normalized score $z(u)$:

$$z(u) = \frac{\widehat{\text{LID}}(u) - \mu_{\widehat{\text{LID}}}}{\sigma_{\widehat{\text{LID}}}}, \tag{7}$$

where $\mu_{\widehat{\text{LID}}}$ and $\sigma_{\widehat{\text{LID}}}$ denote the mean and standard deviation of the set of estimated LID values $\{\widehat{\text{LID}}(v) \mid v \in V\}$ computed across the entire graph. We then formulate Φ using a logistic function to smoothly map the Z-score to the operational range $[\alpha_{\min}, \alpha_{\max}]$:

$$\Phi(\widehat{\text{LID}}(u)) = \alpha_{\min} + \frac{\alpha_{\max} - \alpha_{\min}}{1 + \exp(z(u))}. \tag{8}$$

We employ a logistic function rather than a linear mapping to mitigate the impact of extreme outliers inherent in LID estimations. Because LID distributions are typically heavy tailed, a linear scaling would be severely skewed by a few extreme values, compressing the α assignments for the vast majority of nodes into a narrow, indistinguishable band. The logistic function resolves this by acting as a smooth squashing mechanism. Centered on the Z score, it allocates its steepest and most sensitive region to the bulk of the population near the mean. Simultaneously, it safely clamps outlier nodes to the operational boundaries without distorting the global mapping resolution. In experiments (§4), we set $\alpha_{\min} = 1.0$ and $\alpha_{\max} = 1.5$ following standard practices in graph indexing [36], unless otherwise stated. This formulation ensures that nodes with average complexity ($z(u) \approx 0$) receive an intermediate parameter ($\alpha \approx 1.25$), while those in highly complex regions ($z(u) > 0$) are smoothly penalized toward the strict routing limit of 1.0.

The mapping function Φ satisfies the following geometric properties that are essential for stable graph construction: Monotonicity and Boundedness. We state these properties formally as propositions and provide proofs in the Appendix.

Proposition 3.5 (Monotonicity)**.** *The mapping function Φ is strictly decreasing with respect to the estimated local intrinsic dimensionality. Formally, given that the standard deviation of the LID estimates $\sigma_{\widehat{LID}} > 0$ and the pruning range $\alpha_{\max} > \alpha_{\min}$, the derivative satisfies:*

$$\frac{d\Phi}{d\widehat{LID}(u)} < 0. \tag{9}$$

Proof. See Appendix A.1, Proof A.1. □

Proposition 3.6 (Boundedness)**.** *The pruning parameter $\alpha(u)$ derived from the mapping function is strictly bounded within the prescribed operational interval. That is, for any node u with a finite LID estimate, the following holds:*

$$\alpha_{\min} < \alpha(u) < \alpha_{\max}. \tag{10}$$

Proof. See Appendix A.1, Proof A.2. □

3.3 Indexing Algorithms

The Manifold-Consistent Graph Indexing (MCGI) algorithm introduces a geometric calibration phase to a graph indexing procedure. Unlike static methods that apply a uniform connectivity rule, MCGI executes in two distinct stages to ensure the topology respects the manifold structure, as follows. The full description of the algorithm is in Appendix A.3, Algorithm 2.

Phase 1: Geometric Calibration. Before modifying the graph topology, the system first performs a global analysis of the dataset geometry. We estimate the LID for every point and aggregate the population statistics (μ, σ) defined in Section 3.2. This phase "freezes" the geometric profile of the dataset. By pre-computing these statistics, we decouple the complexity estimation from the graph update loop, ensuring that the mapping function Φ remains stable and computationally efficient during the intensive edge-selection process.

Phase 2: Manifold-Consistent Refinement. The index construction follows an iterative refinement strategy. Let $\mathcal{N}(u)$ denote the set of neighbors for node u in the graph G. In each iteration, the algorithm dynamically updates $\mathcal{N}(u)$ by considering a candidate pool $\mathcal{C}$ obtained from a greedy search on the current graph structure:

1. Queries the pre-computed geometric profile to determine the node-specific constraint $\alpha(u)$.
2. Explores the graph to identify a candidate pool $\mathcal{C}$.
3. Filters connections using the dynamic occlusion criterion.

We analyze the computational complexity as follows. Let N be the total number of vectors in dataset $\mathcal{X}$, R the maximum degree, L the construction beam width, and T the number of refinement iterations. The two phases of MCGI have the following time complexities:

- *Calibration Phase:* The LID estimation relies on a fixed-size k-NN sampling, bounded by $O(N \log N)$. The subsequent parameter mapping is a linear scan $O(N)$.
- *Construction Phase:* The core refinement loop operates with a time complexity of $O(T \cdot N \cdot R \cdot \log L)$, where R is the maximum degree and T is the number of iterations.

Since the calibration is a one-pass pre-processing step (as opposed to the multi-pass iterative refinement in Phase 2), the total time complexity remains dominated by the graph refinement, ensuring that MCGI scales linearly with N, consistent with state-of-the-art methods like DiskANN [36].

In practice, calculating the exact LID for the entire dataset may cause a computational bottleneck on billion-scale data. To address this, we propose *Online-MCGI*, which integrates LID estimation directly into the graph construction process. First, we bootstrap global statistics (μ, σ) using a small random subset of the data, reducing the preprocessing cost to negligible levels. Then, during the iterative graph refinement, we estimate the local LID $\hat{\ell}_u$ for each node u on-the-fly using its current candidate set $\mathcal{C}$ obtained from the greedy search. While the initial estimates may be noisy, they progressively converge to the true manifold dimensionality as the neighbor quality improves with each iteration. This approach allows the pruning parameter α_u to dynamically adapt to local geometric complexity without incurring the prohibitive cost of offline global analysis. The full description of this online algorithm can be found in Appendix A.3, Algorithm 2.

3.4 Theoretical Analysis

3.4.1 Geometric Complexity and Adaptive Routing

The efficiency of greedy routing on a proximity graph is intrinsically tied to the local geometric properties of the underlying data manifold $\mathcal{M}$. Under the Manifold Hypothesis, the search space is locally characterized by the Local Intrinsic Dimensionality (LID) at query q. We first formalize the relationship between this local geometry and the computational cost required to identify the nearest neighbor, as follows.

Lemma 3.7 (Local Complexity Lower Bound)**.** *Consistent with the complexity bounds established for growth-restricted metrics [23], for a query q on a manifold $\mathcal{M}$, the expected number of distance*

evaluations N_{dist} required for successful greedy routing scales exponentially with the local intrinsic dimensionality $d = LID(q)$:

$$\mathbb{E}[N_{dist}] \geq \Omega\left(\frac{1}{\sqrt{d}} \cdot \exp(\lambda \cdot d)\right), \tag{11}$$

where $\lambda > 0$ is a geometric constant derived from the required convergence rate.

Proof. See Appendix A.1, Proof A.3. □

Lemma 3.7 reveals a fundamental geometric barrier: for a fixed budget L, the probability of routing failure grows exponentially in high-LID regions. To address this, MCGI adopts an *Iso-Recall* strategy, aiming to homogenize the search reliability across the heterogeneous manifold.

Proposition 3.8 (Optimal Budget Allocation)**.** *To maintain a uniform bound on the routing failure probability δ across the manifold (i.e., $\mathbb{P}(fail \mid q) \leq \delta$ for all $q \in \mathcal{M}$), the search beam width $L(q)$ must scale exponentially with the local intrinsic dimensionality:*

$$L(q) \propto \exp\left(\lambda \cdot LID(q)\right). \tag{12}$$

Proof. See Appendix A.1, Proof A.4. □

While Proposition 3.8 establishes that the search budget L must theoretically adapt to local geometric complexity, dynamically modulating L per node during online serving disrupts memory alignment and degrades SIMD parallel efficiency. Therefore, in practical system deployments, L is strictly bounded as a constant.

3.4.2 Topological Fidelity and Connectivity

A primary theoretical concern with aggressive edge pruning is the potential fracture of the connectivity backbone [42]. We prove that MCGI guarantees global reachability by strictly preserving the underlying manifold skeleton.

The edge selection in MCGI is governed by the adaptive pruning parameter $\alpha(u)$. Specifically, an edge (u, v) is pruned if there exists a witness node n such that $\alpha(u) \cdot d(n, v) \leq d(u, v)$. This condition defines an *exclusion region* around the midpoint of u and v. As $\alpha(u)$ increases, this region shrinks, making pruning more conservative.

Proposition 3.9 (Connectivity Preservation)**.** *Let G_{EMST} and G_{RNG} denote the Euclidean Minimum Spanning Tree and the Relative Neighborhood Graph, respectively. For any configuration of points in general position, provided that $\alpha(u) \geq 1.0$ for all $u \in V$, the graph G_{MCGI} satisfies the strict inclusion hierarchy:*

$$E_{EMST} \subseteq E_{RNG} \subseteq E_{MCGI}. \tag{13}$$

Consequently, G_{MCGI} is connected.

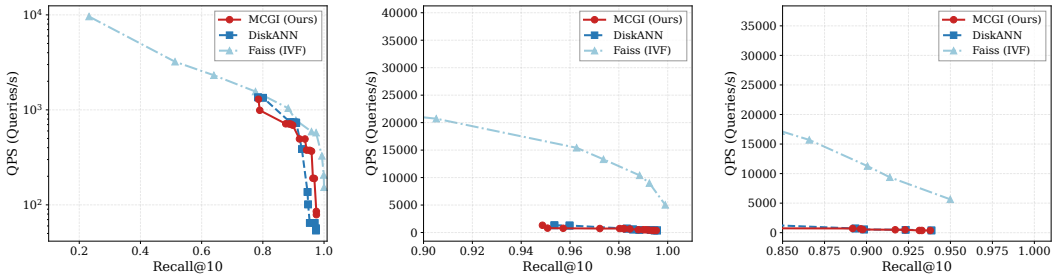
Proof. See Appendix A.1, Proof A.5. □

Proposition 3.9 ensures that there are no structural dead ends. While Lemma 3.7 dictates the *cost* of routing, Proposition 3.9 ensures the *possibility* of routing. Even in the worst-case scenario where heuristics fail, a path exists from the entry point to any target via the edges of the underlying EMST, provided the graph traversal algorithm has sufficient width to discover these backbone links.

4 Evaluation

We implemented MCGI on top of the official C++ codebase of DiskANN [36]. MCGI implementation is open-sourced available at: `https://github.com/`. We focus on answering the following five research questions.

(1) RQ1 (High-Dimensional Effectiveness): How does MCGI perform compared to baselines on high-dimensional data (e.g., GIST1M) where the curse of dimensionality typically degrades the efficiency of graph-based indices?



(a) GIST1M (960-dim) (b) SIFT1M (128-dim) (c) GloVe-100 (100-dim)

Figure 1: Recall-QPS Trade-off. Comparison of MCGI against DiskANN and Faiss (mmap).

(2) RQ2 (High-Recall Efficiency): Can MCGI maintain high query throughput (QPS) under strict recall requirements (e.g., Recall@10 $\geq$ 95%), making it suitable for latency-critical production environments?

(3) RQ3 (Billion-Scale Scalability): Does the proposed method scale effectively to billion-scale datasets, including SIFT1B and T2I-1B, and demonstrate superiority over industry-standard baselines across both I/O-bound and compute-bound workloads?

(4) RQ4 (Resource Efficiency): Does the manifold-aware routing strategy translate to lower I/O overhead and reduced query latency on modern hardware (e.g., NVMe SSDs and AVX2 enabled CPUs)?

(5) RQ5 (Parameter Sensitivity and Ablation): How sensitive is MCGI to the choice of mapping function parameters ($\alpha_{\min}, \alpha_{\max}$)? How does the performance degrade if we disable the adaptive mechanism and use a simpler approach like a linear mapping?

(6) RQ6 (Recall Performance vs. Advanced Graph Indexing): How does MCGI compare to in-memory ANN algorithms like CSPG [41] and PipeANN [12, 11]?

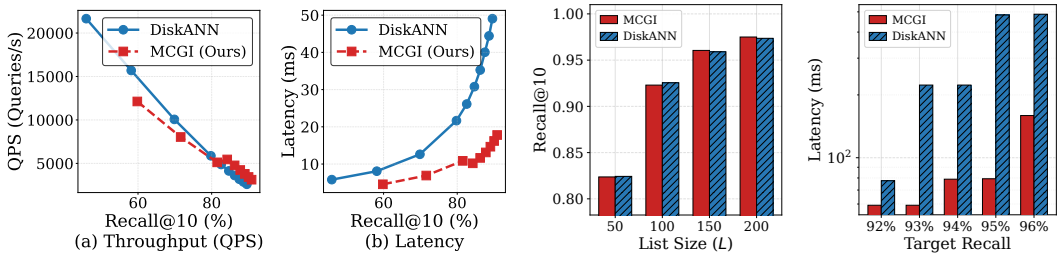
4.1 Experimental Setup

All experiments were conducted on the Chameleon Cloud [25] platform using a compute node equipped with dual-socket Intel Xeon Platinum 8380 CPUs (Ice Lake architecture, 2.30GHz, 80 cores total) and 256 GiB of RAM. To simulate a cost-effective large-scale retrieval scenario, the indices are stored on a single 480 GB Micron 5300 PRO Enterprise SSD. The operating system is Ubuntu 24.04 LTS. All algorithms were compiled using GCC 11.4 with -O3 and AVX2 optimizations enabled to fully utilize the hardware instruction set.

We evaluate our method on five standard benchmarks ranging from million-scale to billion-scale: SIFT1M [21], GloVe-100 [30], GIST1M [21], SIFT1B [22], and T2I-1B [34]. Our evaluation compares MCGI against five state-of-the-art baselines: PipeANN [12, 11], CSPG [41], DiskANN (Vamana) [36], IVF-Flat (Faiss) [20], and HNSW [29]. More details of these datasets and baselines are provided in Appendix A.2.

4.2 Experimental Results

High-Dimensional Effectiveness (RQ1) While the in-memory Faiss baseline establishes a theoretical throughput upper-bound, disk-resident indices face severe constraints from SSD random I/O latency. As illustrated in Figure 1a, MCGI mitigates this bottleneck on the high-dimensional GIST1M dataset by leveraging manifold-aware routing to minimize necessary disk reads, achieving 375 QPS at 95% recall and delivering a 5.8$\times$ speedup over the baseline DiskANN. Furthermore, evaluations on lower-dimensional datasets like SIFT1M and GloVe-100 (Figures 1b and 1c) confirm that MCGI achieves performance parity with DiskANN. These results indicate that our adaptive mechanism robustly reduces to standard greedy search on simpler geometries without incurring any computational overhead, bridging the gap between disk-based scalability and in-memory efficiency.



(a) Billion Scale Performance on SIFT1B (b) Sensitivity (c) Latency

Figure 2: Billion scale scalability and resource efficiency evaluations.

High Recall Efficiency (RQ2) Real world applications typically demand strict accuracy guarantees, such as Recall@10 being greater than or equal to 95%. As summarized in Table 1, while in memory methods like HNSW and IVF Flat establish high performance ceilings of 2,165 QPS and 590.5 QPS respectively, traditional disk resident indices face severe bottlenecks. For instance, the state of the art DiskANN plateaus at 64.7 QPS due to excessive random I/O from static routing in sparse high dimensional spaces. MCGI effectively bridges this divide by delivering 375.1 QPS at 95% recall, outperforming DiskANN by a factor of $5.8\times$ and reaching approximately 64% of the memory resident IVF Flat index performance. This demonstrates that optimized manifold aware I/O management can push disk based indexing speeds into a range previously reserved for in memory structures.

Table 1: Peak QPS at $\geq$ 95% recall.

Algorithm	Locality	QPS
DiskANN [36]	disk	64.7
MCGI (this work)	disk	375.1
IVF Flat [20]	memory	590.5
HNSW [29]	memory	2,165

Billion Scales (RQ3) To validate scalability, we conducted evaluations on billion-scale datasets including SIFT1B. Figure 2a illustrates the recall QPS trade-off and latency comparisons. While DiskANN maintains competitive performance at moderate recall levels, its throughput degrades sharply as retrieval requirements tighten. At a strict recall target of 90%, DiskANN drops to 2,597 QPS and incurs a mean latency of 49.06 ms due to excessive redundant I/O operations. Conversely, MCGI maintains a robust throughput of 3,436 QPS and reduces the mean latency to 16.20 ms, representing a 3x reduction. This confirms that our manifold-consistent pruning strategy effectively identifies optimal neighbors with significantly fewer disk accesses.

Resource Efficiency (RQ4) The search list size parameter L governs the balance between search quality and computational cost. As shown in Figure 2b, MCGI exhibits a recall trajectory that closely mirrors DiskANN across the tested range. This parity demonstrates that our geometry-aware routing is robust, retaining high recall capabilities without degrading search quality under standard L configurations. Furthermore, Figure 2c details the query latency on GIST1M, showing that MCGI significantly reduces tail latency in the high recall range. By minimizing the random I/O required to escape local minima in high-dimensional spaces, MCGI translates manifold-aware routing into tangible lower I/O overhead for resource constrained deployments.

Ablation and Parameter Sensitivity (RQ5) We evaluate the sensitivity of our proposed routing mechanism to the dynamic pruning upper bound, α_{max}, which governs the trade-off between graph connectivity and distance computation overhead. As depicted in Figure 3(a), tuning α_{max} from 1.2 to 1.7 provides a smooth and predictable Pareto frontier for the QPS-Recall tradeoff. While a lower bound ($\alpha_{max} = 1.2$) limits connectivity, increasing the bound allows MCGI to preserve critical manifold paths that are otherwise prematurely pruned. Specifically, at $\alpha_{max} = 1.7$, MCGI successfully breaks the baseline's recall ceiling (reaching 96.45%) while maintaining more than double the search throughput (>130 QPS) of the baseline. This progression demonstrates that our dynamic alpha strategy effectively balances topological integrity with computational efficiency. To further validate the mathematical necessity of our non-linear density mapping, we conduct an ablation study comparing our proposed LID-aware Sigmoid transformation against a vanilla Linear mapping. As shown in Figures 3(b)-(d) across three distinct topological configurations, replacing the Sigmoid function with a naive Linear scaling results in a severe performance collapse in the extreme high-recall

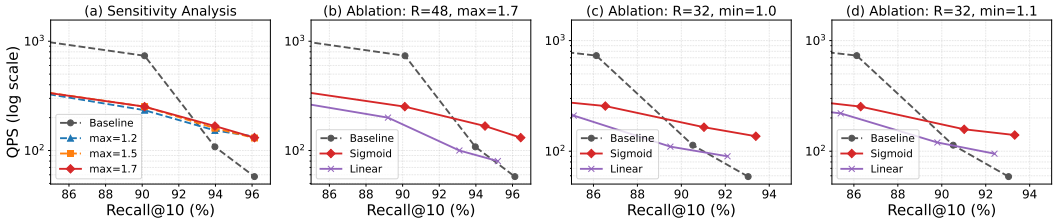


Figure 3: Ablation and sensitivity analysis of the mapping function and pruning parameters.

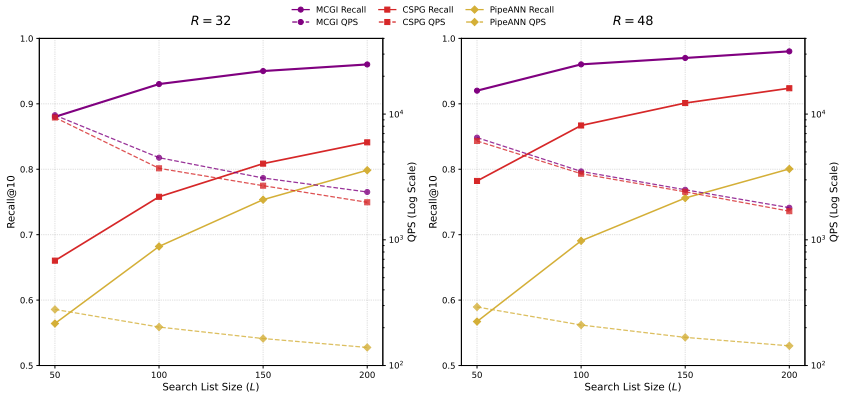


Figure 4: Performance comparison between MCGI, CSPG, and PipeANN on GIST1M under fixed degree constraints.

setup. For instance, under the $R = 48$ configuration, the Linear mapping suffers a nearly 50% drop in QPS and a noticeable degradation in recall compared to the Sigmoid variant. This confirms our theoretical analysis: high-dimensional manifolds exhibit localized, extreme density variations. A rigid linear transformation fails to adequately buffer these spikes, leading to indiscriminate edge pruning or redundant distance calculations. The Sigmoid function smoothly mitigates these topological irregularities, proving indispensable for breaking the baseline's accuracy ceiling without sacrificing query latency.

Comparison with Advanced Graph Baselines (RQ6) To position MCGI within the landscape of state-of-the-art graph indexing, we conduct a head-to-head comparison with CSPG [41] and PipeANN [11, 12]. As illustrated in Figure 4, we evaluate these systems under identical storage budgets by fixing the maximum out-degree at $R = 32$ and $R = 48$. The results reveal a fundamental performance gap between MCGI and existing baselines on the high-dimensional GIST1M manifold. Specifically, while PipeANN employs aggressive I/O pipeline parallelism via io_uring to mask SSD latency, its search performance suffers a catastrophic collapse due to a fragmented topology. Under the $R = 32$ configuration, PipeANN struggles to surpass an 80% recall ceiling even with an enlarged search pool ($L = 200$), incurring excessive I/O overhead (averaging > 220 IOs per query). In contrast, MCGI achieves a near-perfect recall of 96.02% under the same constraint, outperforming PipeANN in both accuracy and throughput by a substantial margin.

5 Conclusion

We presented Manifold Consistent Graph Indexing (MCGI), a new indexing algorithm that fundamentally rethinks vector search by aligning graph routing with the intrinsic geometry of high-dimensional data. By resolving the discrepancy between Euclidean and geodesic distances, MCGI provides robust theoretical guarantees on connectivity while effectively mitigating the curse of dimensionality. Comprehensive evaluations on billion-scale benchmarks, including SIFT1B and T2I-1B, confirm the versatility of our approach across both I/O-intensive and compute-intensive scenarios. Our results demonstrate that MCGI reduces query latency by up to 3x compared to the state-of-the-art DiskANN, delivering a throughput that rivals the performance of in-memory inverted indices.

Acknowledgments and Disclosure of Funding

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation. The authors would like to thank Microsoft Research for merging some of the MCGI source code into the C++ main branch of DiskANN in March 2026.

References

[1] Laurent Amsaleg, Oussama Chelly, Teddy Furon, Stéphane Girard, Michael E. Houle, Ken-ichi Kawarabayashi, and Michael Nett. Estimating local intrinsic dimensionality. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '15, page 29–38, New York, NY, USA, 2015. Association for Computing Machinery.

[2] RICHARD BELLMAN and Stuart Dreyfus. *Dynamic Programming*, volume 33. Princeton University Press, 2010.

[3] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Commun. ACM*, 18(9):509–517, September 1975.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[5] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. Spann: highly-efficient billion-scale approximate nearest neighbor search. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NIPS '21, Red Hook, NY, USA, 2021. Curran Associates Inc.

[6] Sanjoy Dasgupta and Yoav Freund. Random projection trees and low dimensional manifolds. In *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing*, STOC '08, page 537–546, New York, NY, USA, 2008. Association for Computing Machinery.

[7] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry*, SCG '04, page 253–262, New York, NY, USA, 2004. Association for Computing Machinery.

[8] Haya Diwan, Jinrui Gou, Cameron Musco, Christopher Musco, and Torsten Suel. Navigable graphs for high-dimensional nearest neighbor search: Constructions and limits. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 59513–59531. Curran Associates, Inc., 2024.

[9] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.*, 12(5):461–474, January 2019.

[10] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization. volume 36, page 744–755, USA, April 2014. IEEE Computer Society.

[11] Hao Guo and Youyou Lu. Achieving low-latency graph-based vector search via aligning best-first search algorithm with ssd. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 171–186, Boston, MA, 2025. USENIX Association.

[12] Hao Guo and Youyou Lu. Odinann: Direct insert for consistently stable performance in billion-scale graph-based vector search. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, Santa Clara, CA, 2026. USENIX Association.

[13] A. Gupta, R. Krauthgamer, and J.R. Lee. Bounded geometries, fractals, and low-distortion embeddings. In *44th Annual IEEE Symposium on Foundations of Computer Science, 2003. Proceedings.*, pages 534–543, 2003.

[14] Antonin Guttman. R-trees: a dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, SIGMOD '84, page 47–57, New York, NY, USA, 1984. Association for Computing Machinery.

[15] Junfeng He, Sanjiv Kumar, and Shih-Fu Chang. On the difficulty of nearest neighbor search. In *Proceedings of the 29th International Coference on International Conference on Machine Learning*, ICML'12, page 41–48, Madison, WI, USA, 2012. Omnipress.

[16] Michael E. Houle. Local intrinsic dimensionality I: an extreme-value-theoretic foundation for similarity applications. In Christian Beecks, Felix Borutta, Peer Kröger, and Thomas Seidl, editors, *Similarity Search and Applications - 10th International Conference, SISAP 2017, Munich, Germany, October 4-6, 2017, Proceedings*, volume 10609 of *Lecture Notes in Computer Science*, pages 64–79. Springer, 2017.

[17] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, STOC '98, page 604–613, New York, NY, USA, 1998. Association for Computing Machinery.

[18] Piotr Indyk and Haike Xu. Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 66239–66256. Curran Associates, Inc., 2023.

[19] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023.

[20] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[21] Herve Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[22] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. Searching in one billion vectors: Re-rank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 861–864, 2011.

[23] David R. Karger and Matthias Ruhl. Finding nearest neighbors in growth-restricted metrics. In *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing*, STOC '02, page 741–750, New York, NY, USA, 2002. Association for Computing Machinery.

[24] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics.

[25] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

[26] Elizaveta Levina and Peter Bickel. Maximum likelihood estimation of intrinsic dimension. In L. Saul, Y. Weiss, and L. Bottou, editors, *Advances in Neural Information Processing Systems*, volume 17. MIT Press, 2004.

[27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[28] Xingjun Ma, Bo Li, Yisen Wang, Sarah M. Erfani, Sudanthi N. R. Wijewickrema, Michael E. Houle, Grant Schoenebeck, Dawn Song, and James Bailey. Characterizing adversarial subspaces using local intrinsic dimensionality. In *International Conference on Learning Representations*, 2018.

[29] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. volume 42, page 824–836, USA, April 2020. IEEE Computer Society.

[30] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*, pages 1532–1543. ACL, 2014.

[31] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 2021.

[32] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. In *Foundations and Trends in Information Retrieval*, volume 3, pages 333–389. Now Publishers, Inc., 2009.

[33] Sam T. Roweis and Lawrence K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.

[34] Harsha Vardhan simhadri, Martin Aumüller, Matthijs Douze, Dmitry Baranchuk, Amir Ingber, Edo Liberty, George Williams, Ben Landrum, Magdalen Dobson Manohar, Mazin Karjikar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Weiguo Zheng, Yizhuo Chen, Jie Yin, and Ben Huang. Results of the big ANN: NeurIPS'23 competition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.

[35] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search. In *NeurIPS 2021 Competitions and Demonstrations Track*, pages 177–189. PMLR, 2022.

[36] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. Diskann: fast accurate billion-point nearest neighbor search on a single node. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, 2019.

[37] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.

[38] Godfried T. Toussaint. The relative neighbourhood graph of a finite planar set. *Pattern Recognition*, 12(4):261–268, 1980.

[39] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. 2023.

[40] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and Wei Wang. Steiner-hardness: A query hardness measure for graph-based ann indexes. *Proc. VLDB Endow.*, 17(13):4668–4682, September 2024.

[41] Ming Yang, Yuzheng Cai, and Weiguo Zheng. Cspg: Crossing sparse proximity graphs for approximate nearest neighbor search. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 103076–103100. Curran Associates, Inc., 2024.

[42] Dongfang Zhao and Li Yang. Incremental isometric embedding of high-dimensional data using connected neighborhood graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(1):86–98, 2009.

[43] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. Towards efficient index construction and approximate nearest neighbor search in high-dimensional spaces. *Proc. VLDB Endow.*, 16(8):1979–1991, April 2023.

A Technical appendices and supplementary material

A.1 Complete Proofs

Proposition A.1 (Monotonicity)**.** *The mapping function Φ is strictly decreasing with respect to the estimated local intrinsic dimensionality. Formally, given that the standard deviation of the LID estimates $\sigma_{\widehat{LID}} > 0$ and the pruning range $\alpha_{\max} > \alpha_{\min}$, the derivative satisfies:*

$$\frac{d\Phi}{d\widehat{LID}(u)} < 0. \tag{14}$$

Proof. Let $L = \widehat{\mathrm{LID}}(u)$ be the independent variable. We define the normalized Z-score z as a function of L:

$$z(L) = \frac{L - \mu_{\widehat{\mathrm{LID}}}}{\sigma_{\widehat{\mathrm{LID}}}}. \tag{15}$$

The mapping function is defined as:

$$\Phi(L) = \alpha_{\min} + \frac{C}{1 + \exp(z(L))}, \tag{16}$$

where $C = \alpha_{\max} - \alpha_{\min}$. Since we strictly require $\alpha_{\max} > \alpha_{\min}$, it follows that $C > 0$. To determine the sign of the gradient, we apply the chain rule:

$$\frac{d\Phi}{dL} = \frac{d\Phi}{dz} \cdot \frac{dz}{dL}. \tag{17}$$

First, we differentiate the Z-score term with respect to L:

$$\frac{dz}{dL} = \frac{1}{\sigma_{\widehat{\mathrm{LID}}}}. \tag{18}$$

Next, we differentiate the logistic component Φ with respect to z:

$$\frac{d\Phi}{dz} = \frac{d}{dz}\left(\alpha_{\min} + C(1 + e^z)^{-1}\right) \tag{19}$$

$$= C \cdot (-1) \cdot (1 + e^z)^{-2} \cdot \frac{d}{dz}(1 + e^z) \tag{20}$$

$$= -C \cdot \frac{e^z}{(1 + e^z)^2}. \tag{21}$$

Combining these terms yields the full derivative:

$$\frac{d\Phi}{dL} = -\frac{C}{\sigma_{\widehat{\mathrm{LID}}}} \cdot \frac{e^z}{(1 + e^z)^2}. \tag{22}$$

We analyze the sign of each component:

- The operational range constant $C > 0$.
- The standard deviation $\sigma_{\widehat{\text{LID}}} > 0$, assuming the dataset exhibits non-zero geometric variance.
- The exponential function $e^z > 0$ for all $z \in \mathbb{R}$.
- The denominator $(1 + e^z)^2 > 0$.

Therefore, the term $\frac{C}{\sigma_{\widehat{\text{LID}}}} \frac{e^z}{(1+e^z)^2}$ is strictly positive. The leading negative sign guarantees that $\frac{d\Phi}{dL} < 0$. This confirms that the pruning parameter α strictly decreases as the local geometric complexity increases, thereby enforcing a more conservative graph topology in high-LID regions to prevent topology distortion in complex regions. □

Proposition A.2 (Boundedness)**.** *The pruning parameter $\alpha(u)$ derived from the mapping function is strictly bounded within the prescribed operational interval. For any node u with a finite LID estimate:*

$$\alpha_{\min} < \alpha(u) < \alpha_{\max}. \tag{23}$$

Proof. Let $S(u)$ denote the logistic component of the mapping function:

$$S(u) = \frac{1}{1 + \exp(z(u))}. \tag{24}$$

For any finite input $\widehat{\text{LID}}(u)$, the Z-score $z(u)$ is finite. The exponential function maps the real line to the positive real line, i.e., $\exp(z(u)) \in (0, \infty)$. Consequently, the denominator lies in the interval $(1, \infty)$. Taking the reciprocal yields the bounds for the logistic component:

$$0 < S(u) < 1. \tag{25}$$

Substituting $S(u)$ back into the definition of Φ:

$$\alpha(u) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot S(u). \tag{26}$$

Since $(\alpha_{\max} - \alpha_{\min}) > 0$, we can apply the inequality boundaries:

$$\alpha(u) > \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot 0 = \alpha_{\min}, \tag{27}$$

$$\alpha(u) < \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot 1 = \alpha_{\max}. \tag{28}$$

This proves that the topology is strictly confined. The pruning behavior never exceeds the relaxation upper limit ($\alpha_{\max}$) and never becomes stricter than the lower limit ($\alpha_{\min}$), ensuring graph connectivity and preventing degree explosion. □

Lemma A.3 (Local Complexity Lower Bound)**.** *Consistent with the complexity bounds established for growth-restricted metrics [23], for a query q on a manifold $\mathcal{M}$, the expected number of distance evaluations N_{dist} required for successful greedy routing scales exponentially with the local intrinsic dimensionality $d = LID(q)$:*

$$\mathbb{E}[N_{dist}] \geq \Omega\left(\frac{1}{\sqrt{d}} \cdot \exp(\lambda \cdot d)\right), \tag{29}$$

where $\lambda > 0$ is a geometric constant derived from the required convergence rate.

Proof. Let the local dataset around a query q be modeled as a uniform distribution on a manifold of intrinsic dimension d. Consider a greedy routing step where the algorithm is currently at a node u with distance $r = \|u - q\|$.

1. Geometric Condition for Improvement. To make progress, the algorithm must identify a neighbor v such that the distance to the query is reduced by a factor ϵ (where $0 < \epsilon < 1$), i.e., $\|v - q\| \leq (1 - \epsilon)r$. Let $s = \|u - v\|$ be the distance between the current node and the candidate neighbor (the step size). Applying the Law of Cosines to triangle $\triangle uvq$, we have:

$$\|v - q\|^2 = r^2 + s^2 - 2rs\cos\theta, \tag{30}$$

where θ is the angle between the vector $\vec{uq}$ and $\vec{uv}$. The improvement condition $\|v-q\|^2 \leq (1-\epsilon)^2 r^2$ implies:

$$\begin{aligned} r^2 + s^2 - 2rs\cos\theta &\leq (1-\epsilon)^2 r^2, \\ 2rs\cos\theta &\geq r^2 + s^2 - r^2(1-\epsilon)^2, \\ \cos\theta &\geq \frac{r^2 + s^2 - r^2(1-\epsilon)^2}{2rs}. \end{aligned} \tag{31}$$

Assuming the step size is small relative to the distance (i.e., $s \ll r$), the right-hand side is dominated by $1-(1-\epsilon)^2 \approx 2\epsilon$, yielding a critical angle threshold θ_{max} such that any successful neighbor must fall within the cone defined by $0 \leq \theta \leq \theta_{max}$.

2. Volume Concentration of the Improving Region. In the tangent space at u, the directions of neighbors are uniformly distributed on the unit hypersphere $\mathbb{S}^{d-1}$. The probability $P_{success}$ of finding a neighbor in the improving cone corresponds to the ratio of the surface area of the spherical cap $\mathcal{C}(\theta_{max})$ to the total surface area of $\mathbb{S}^{d-1}$. This ratio is given by the regularized incomplete beta function, which can be expressed in terms of the integral over the polar angle ϕ:

$$P_{success} = \frac{\text{Area}(\mathcal{C}(\theta_{max}))}{\text{Area}(\mathbb{S}^{d-1})} = \frac{\int_0^{\theta_{max}} \sin^{d-2}\phi \, d\phi}{\int_0^{\pi} \sin^{d-2}\phi \, d\phi}. \tag{32}$$

3. Asymptotic Analysis. We analyze the asymptotic behavior of this ratio as $d \to \infty$. The denominator relates to the surface area of the hypersphere and can be evaluated using the standard integral identity for powers of sine. Exploiting the symmetry of $\sin\phi$ around $\pi/2$, we have:

$$\begin{aligned} \int_0^{\pi} \sin^{d-2}\phi \, d\phi &= 2\int_0^{\frac{\pi}{2}} \sin^{d-2}\phi \, d\phi \\ &= 2 \cdot \frac{\sqrt{\pi}}{2} \frac{\Gamma(\frac{d-1}{2})}{\Gamma(\frac{d}{2})} = \sqrt{\pi}\frac{\Gamma(\frac{d-1}{2})}{\Gamma(\frac{d}{2})}. \end{aligned} \tag{33}$$

For high-dimensional spaces ($d \gg 1$), we apply the asymptotic property of the Gamma function ratio, $\frac{\Gamma(x+a)}{\Gamma(x)} \approx x^a$. Setting $x = d/2$ and $a = -1/2$, we obtain the approximation:

$$\sqrt{\pi}\frac{\Gamma(\frac{d}{2}-\frac{1}{2})}{\Gamma(\frac{d}{2})} \approx \sqrt{\pi}\left(\frac{d}{2}\right)^{-\frac{1}{2}} = \sqrt{\frac{2\pi}{d}}. \tag{34}$$

For the numerator, we exploit the concentration of measure near the integration boundary θ_{max}. Since the integrand $\sin^{d-2}\phi$ decays exponentially fast away from θ_{max}, the integral is dominated by the contribution near the upper limit. We apply integration by parts to extract the leading order term. Recall that $\frac{d}{d\phi}(\sin^{d-1}\phi) = (d-1)\sin^{d-2}\phi\cos\phi$. We rewrite the integrand as:

$$\int_0^{\theta_{max}} \sin^{d-2}\phi \, d\phi = \int_0^{\theta_{max}} \frac{1}{(d-1)\cos\phi}\frac{d}{d\phi}(\sin^{d-1}\phi) \, d\phi. \tag{35}$$

Neglecting lower-order terms generated by the derivative of $\frac{1}{\cos\phi}$, the integral approximates to the boundary term:

$$\left[\frac{\sin^{d-1}\phi}{(d-1)\cos\phi}\right]_0^{\theta_{max}} = \frac{\sin^{d-1}\theta_{max}}{(d-1)\cos\theta_{max}}. \tag{36}$$

Substituting these approximations back into the probability ratio, we obtain:

$$P_{success} \approx \frac{1}{\sqrt{2\pi}\cos\theta_{max}} \cdot \frac{\sqrt{d}}{d-1} \cdot (\sin\theta_{max})^{d-1}. \tag{37}$$

For large d, the term $\frac{\sqrt{d}}{d-1}$ is asymptotically equivalent to $d^{-1/2}$. Since θ_{max} is fixed by the geometric constraints, the trigonometric factors are constant relative to d. Let $\gamma = \sin\theta_{max} < 1$. The probability scales as:

$$P_{success} \propto d^{-1/2} \cdot \gamma^d = d^{-1/2}\exp(d\ln\gamma). \tag{38}$$

Defining the decay rate $\lambda = -\ln\gamma > 0$ (since $\gamma < 1$), we strictly obtain:

$$P_{success} \propto d^{-1/2}\exp(-\lambda \cdot d). \tag{39}$$

4. Complexity Bound. The expected number of distance evaluations follows $\mathbb{E}[N_{dist}] = 1/P_{success}$. Thus:

$$\mathbb{E}[N_{dist}] \geq \Omega\left(\sqrt{d} \cdot \exp(\lambda \cdot d)\right). \tag{40}$$

This confirms that the routing cost is dominated by the exponential term $\exp(\lambda \cdot \text{LID}(q))$, necessitating the adaptive beam width design in MCGI. □

Proposition A.4 (Optimal Budget Allocation)**.** *To maintain a uniform bound on the routing failure probability δ across the manifold (i.e., $\mathbb{P}(\text{fail} \mid q) \leq \delta$ for all $q \in \mathcal{M}$), the search beam width $L(q)$ must scale exponentially with the local intrinsic dimensionality:*

$$L(q) \propto \exp\left(\lambda \cdot \textit{LID}(q)\right). \tag{41}$$

Proof. Consider a beam search with width L. The probability of failing to find a descent step in one round is approximately $(1 - P_{success})^L$. To guarantee a recall target, we enforce a constant upper bound δ on the failure probability:

$$(1 - P_{success})^L \leq \delta. \tag{42}$$

Taking the natural logarithm and using the approximation $\ln(1 - x) \approx -x$ for small $P_{success}$, we require:

$$-L \cdot P_{success} \leq \ln \delta \implies L \geq \frac{-\ln \delta}{P_{success}}. \tag{43}$$

Substituting the relationship $P_{success} = 1/\mathbb{E}[N_{dist}]$ established in Lemma 3.7, we obtain the asymptotic upper bound $P_{success} \leq \mathcal{O}(\sqrt{d} \cdot \exp(-\lambda \cdot d))$, where $d = \text{LID}(q)$. To satisfy the recall constraint in the worst case scenario, the search budget L must grow inversely to $P_{success}$:

$$L \geq \frac{-\ln \delta}{\mathcal{O}(\sqrt{d} \cdot \exp(-\lambda \cdot d))} = \Omega\left(\frac{1}{\sqrt{d}} \cdot \exp(\lambda \cdot d)\right). \tag{44}$$

In high-dimensional spaces ($d \gg 1$), the exponential growth term strictly dominates the polynomial decay factor $1/\sqrt{d}$. By absorbing the lower-order polynomial variations and the confidence factor $-\ln \delta$ into a generalized constant C_δ, the necessary budget asymptotically simplifies to:

$$L(q) \geq C_\delta \cdot \exp(\lambda \cdot \text{LID}(q)). \tag{45}$$

□

Proposition A.5 (Connectivity Preservation)**.** *Let G_{EMST} and G_{RNG} denote the Euclidean Minimum Spanning Tree and the Relative Neighborhood Graph, respectively. For any configuration of points in general position, provided that $\alpha(u) \geq 1.0$ for all $u \in V$, the graph G_{MCGI} satisfies the strict inclusion hierarchy:*

$$E_{EMST} \subseteq E_{RNG} \subseteq E_{MCGI}. \tag{46}$$

Consequently, G_{MCGI} is connected.

Proof. The proof proceeds in two steps. First, we invoke the classical result established by Toussaint [38], which proves that the Relative Neighborhood Graph (RNG) is a supergraph of the Euclidean Minimum Spanning Tree (EMST) for any finite set of points in a metric space:

$$E_{EMST} \subseteq E_{RNG}. \tag{47}$$

Since the EMST connects all vertices in a single component, G_{RNG} is necessarily connected.

Second, we show that $E_{RNG} \subseteq E_{MCGI}$. In the RNG, an edge between u and v is pruned if and only if a witness node falls within the intersection of two spheres centered at u and v with radius $d(u, v)$. We denote this geometric intersection as the exclusion region $\mathcal{R}_{RNG}$. Therefore, an edge exists in the RNG if and only if $\mathcal{R}_{RNG}$ contains no data points. In MCGI, the pruning condition requires the witness node to be within a distance of $\frac{1}{\alpha(u)} d(u, v)$ from v. Because the algorithm enforces $\alpha(u) \geq 1.0$, the pruning radius in MCGI is strictly less than or equal to the pruning radius of the RNG. Consequently, the exclusion region of MCGI is a geometric subset of the RNG exclusion region:

$$\mathcal{R}_{MCGI} \subseteq \mathcal{R}_{RNG}. \tag{48}$$

This subset relationship of exclusion regions creates an inverse superset relationship for the preserved edges. If an edge survives the RNG condition, its $\mathcal{R}_{RNG}$ is empty. Since $\mathcal{R}_{MCGI}$ is contained entirely within $\mathcal{R}_{RNG}$, it must also be empty. Thus, any edge present in the RNG is guaranteed to be preserved in MCGI, yielding the inclusion hierarchy of edge sets:

$$E_{RNG} \subseteq E_{MCGI}. \tag{49}$$

By combining this with the classical result that $E_{EMST} \subseteq E_{RNG}$ derived in [38], we establish the complete global connectivity chain. □

A.2 Additional Experimental Details

Datasets. We evaluate our method on four standard benchmarks, ranging from million-scale to billion-scale, to comprehensively test robustness across varying intrinsic dimensionalities and data volumes. In the following descriptions, N denotes the number of base vectors and D represents the vector dimensionality:

- SIFT1M [21] ($N = 10^6, D = 128$): A standard computer vision dataset using Euclidean distance (L_2).
- GloVe-100 [30] ($N = 1.2 \times 10^6, D = 100$): Word embedding vectors measuring semantic similarity. Following standard practice, we normalize the vectors to unit length and use Euclidean distance as a proxy for Cosine similarity.
- GIST1M [21] ($N = 10^6, D = 960$): A high-dimensional dataset representing global image features. This dataset is particularly challenging for index structures due to the sparsity of the space and the high intrinsic dimensionality.
- SIFT1B [22] ($N = 10^9, D = 128$): A billion-scale dataset used to evaluate the scalability and I/O efficiency of our method. It shares the same feature distribution as SIFT1M but scales the volume by three orders of magnitude, representing a realistic industrial scenario.
- T2I-1B [34] ($N = 10^9, D = 200$): A billion-scale dataset consisting of CLIP [31] embeddings derived from the LAION-5B dataset. Unlike visual descriptors like SIFT, T2I-1B represents a cross-modal retrieval scenario with a more complex manifold structure and higher intrinsic dimensionality, posing greater challenges for graph-based indexing algorithms.

Baselines. We compare MCGI against two baselines, each serving a distinct role:

- PipeANN [11, 12]: A recent state-of-the-art disk-based system that optimizes search throughput via aggressive I/O pipeline parallelism. We include it to evaluate whether system-level asynchronous I/O optimizations can compensate for topological deficiencies in high-dimensional spaces.
- CSPG [41]: A high-performance C++ implementation of proximity graphs designed for efficiency. It serves as a rigorous algorithmic baseline to benchmark our LID-aware pruning strategy against standard graph construction techniques.
- DiskANN (Vamana) [36]: The standard disk-based graph index in Faiss. Since MCGI builds upon the Vamana architecture, this comparison isolates the specific gains derived from our manifold-aware routing strategy.
- IVF-Flat (Faiss) [20]: An in-memory inverted index serving as a performance roofline. Although running in memory-mapped mode, its sequential scanning pattern allows aggressive OS caching, effectively simulating in-memory performance. We include it to benchmark how closely our disk-resident solution approaches the throughput limits of memory-resident systems.
- HNSW [29]: The state-of-the-art in-memory graph index known for its superior recall-latency trade-off. Unlike disk-resident approaches, HNSW maintains the entire hierarchical graph topology in DRAM, completely eliminating I/O latency. We include it to establish a performance ceiling for graph-based search, allowing us to quantify the gap between our disk-resident solution and the theoretical limit of unconstrained memory-resident traversal.

Evaluation Metrics. We adhere to the standard evaluation protocol for ANN search. We measure Recall@10 against QPS (Queries Per Second). Additionally, we report the query latency at critical high-recall operating points (e.g., 95% Recall) to assess the tail latency characteristics.

Build Configuration Table 2 summarizes the dataset specifications and indexing parameters used during the construction of MCGI and baseline DiskANN indices. R denotes the maximum outdegree of each node in the graph. L_{build} is the size of the candidate list during index construction. α Range indicates the range of pruning aggressiveness explored during the LID-based Sigmoid mapping. m_{PQ} specifies the number of bytes allocated for Product Quantization (PQ) codes when applicable. Data Type indicates whether the dataset vectors are stored as 32-bit floating-point numbers or 8-bit unsigned integers.

Table 2: Dataset Specifications and Indexing Parameters.

Dataset	R	L_{build}	α Range	m_{PQ} (Byte)	Data Type
SIFT1M	64	100	$[1.0, 1.5]$	N/A	float32
GloVe-100	64	100	$[1.0, 1.5]$	N/A	float32
GIST1M	96	150	$[1.0, 1.5]$	N/A	float32
SIFT1B	32	50	$[1.0, 1.5]$	16	uint8
T2I-1B	32	50	$[1.0, 1.5]$	16	float32

Search Configuration Table 3 lists the statistical parameters derived from the dataset's LID distribution used in the Sigmoid mapping function $\Phi(\cdot)$, alongside the runtime search sweep parameters. μ_{LID} and σ_{LID} denote the mean and standard deviation of the Local Intrinsic Dimensionality (LID) estimates across the dataset. Beam (L_{search}) indicates the range of candidate list sizes evaluated during query execution to capture the Recall-QPS trade-off. Threads specifies the number of parallel threads utilized during search to maximize hardware concurrency.

Table 3: Runtime Search Parameters and LID Statistics.

Dataset	μ_{LID} (Mean)	σ_{LID} (Std)	Beam (L_{search})	Threads
SIFT1M	14.2	3.1	$10 \rightarrow 100$	1
GloVe-100	18.5	4.2	$10 \rightarrow 120$	1
GIST1M	22.1	5.8	$20 \rightarrow 200$	1
SIFT1B	19.5	7.9	$10 \rightarrow 400$	80
T2I-1B	18.3	7.0	$10 \rightarrow 200$	80

More Experimental Results We further evaluated MCGI on the T2I-1B dataset. Unlike SIFT1B, T2I-1B introduces a distinct challenge characterized by higher dimensionality ($D = 200$) and 32-bit floating-point precision. These factors make distance computations significantly more expensive, shifting the system bottleneck from pure I/O latency towards CPU computation. In the critical high-recall regime (Recall@10 $> 90\%$), MCGI demonstrates a clear advantage over DiskANN, delivering higher QPS at reduced latency. Notably, this performance gain persists even though the I/O reduction is less pronounced than on SIFT1B. The advantage stems from MCGI's geometric pruning strategy: by identifying simpler local manifolds, our algorithm aggressively prunes redundant edges, reducing the average node degree without compromising navigability. This optimization translates directly to fewer expensive floating-point operations per hop, highlighting MCGI's versatility in handling both I/O-bound (SIFT1B) and compute-bound (T2I-1B) billion-scale workloads.

A.3 Full Algorithms

Algorithm 1 outlines the offline construction process of the indexing structure. The procedure is divided into two sequential phases.

- The first phase focuses on geometric calibration by calculating the Local Intrinsic Dimensionality for all vectors in the dataset in parallel. After obtaining the global mean and standard deviation of these estimates, the algorithm assigns a personalized pruning parameter α_u to

each node. This parameter is derived using a logistic function that maps the standardized geometric complexity of the local neighborhood into a bounded operational range.

- The second phase handles topology refinement. Starting from a random graph, the algorithm iteratively optimizes the connections. For each node, it executes a greedy search to identify a pool of candidate neighbors. It then evaluates these candidates alongside existing neighbors using a dynamic occlusion criterion. A candidate is pruned if a previously accepted neighbor falls within a restricted exclusion region defined by the precomputed α_u. Survivors are added to the updated neighbor list until the degree limit is reached.

Algorithm 1 Manifold-Consistent Graph Indexing (MCGI)

```
Input: Dataset X, Max Degree R, Beam Width L
Output: Optimized Graph G

// Phase 1: Geometric Calibration
L ← ParallelEstimateLID(X)
μ ← Mean(L)
σ ← StdDev(L)
for each node u ∈ V in parallel do
  z_u ← (L[u] − μ)/σ
  α_u ← α_min + (α_max − α_min)/(1 + exp(z_u))
end for

// Phase 2: Topology Refinement
G ← RandomGraph(X, R)
for iter ← 1 to MaxIter do
  for each node u ∈ G in parallel do
    C ← GreedySearch(u, G, L)
    N_new ← ∅
    for v ∈ SortByDistance(C ∪ N(u)) do
      pruned ← False
      for n ∈ N_new do
        if α_u · d(n, v) ≤ d(u, v) then
          pruned ← True; break
        end if
      end for
      if not pruned and |N_new| < R then
        N_new.add(v)
      end if
    end for
    N(u) ← N_new
  end for
end for
```

Algorithm 2 presents an optimized online variant designed to handle massive datasets where exhaustive geometric profiling is computationally prohibitive.

- It begins with a fast approximation phase that samples a small subset of the data. By evaluating this subset, the algorithm quickly bootstraps the global mean and standard deviation of the underlying probability distribution. This step provides a coarse but efficient estimate of the overall geometric complexity, enabling the algorithm to initialize the pruning parameters without incurring the cost of a full dataset pass.
- The second phase merges the geometric estimation directly into the iterative topology refinement loop. Instead of computing all complexities beforehand, the algorithm dynamically estimates the node specific dimensionality on the fly using the candidate set discovered during the greedy search step. This real time estimate is immediately used to calculate the specific pruning parameter α_u. The algorithm then applies this dynamic parameter to the pruning criterion, determining which edges to preserve and which to discard. This approach

ensures the graph connectivity adapts to the manifold geometry progressively as the quality of the candidate neighbors improves with each iteration.

Algorithm 2 Online Manifold-Consistent Graph Indexing

```
Input: Dataset X, Max Degree R, Beam Width L, Sample Rate S
Output: Optimized Graph G

// Phase 1: Bootstrap Statistics (Fast Approximation)
X_sample ← RandomSample(X, S)
L_sample ← EstimateLID(X_sample) {Compute LID on subset}
μ ← Mean(L_sample), σ ← StdDev(L_sample)

// Phase 2: Online Topology Refinement
G ← RandomGraph(X, R)
for iter ← 1 to MaxIter do
  for each node u ∈ G in parallel do
    C ← GreedySearch(u, G, L)
    // Online LID Estimation & Parameter Adaptation
    ℓ̂_u ← ComputeLID(u, C) {Estimate from neighbors}
    z_u ← (ℓ̂_u − μ)/σ
    α_u ← α_min + (α_max − α_min)/(1 + exp(z_u))
    N_new ← ∅
    for v ∈ SortByDistance(C ∪ N(u)) do
      pruned ← False
      for n ∈ N_new do
        // Pruning with dynamic α_u
        if α_u · d(n, v) ≤ d(u, v) then
          pruned ← True; break
        end if
      end for
      if not pruned and |N_new| < R then
        N_new.add(v)
      end if
    end for
    N(u) ← N_new
  end for
end for
```